Week 3 Deliverable: Software Design Document

Student: Katleho Ernest Borotho

Student Number: 202322762

1. Introduction

This document presents the software design of the Moderated Learning Management System (LMS). It covers the system architecture, detailed class design, behavioural models and database design. The design is based on the requirements specified in the Week 2 Software Requirements Specification (SRS).

Version 1.0 will be implemented in C++17 as a desktop application with a graphical user interface (GUI) and a local SQLite database. The application will operate on a single school computer without an Internet connection for normal use. Whole-class extensions are excluded from v1.0 and will be addressed in Change Request CR1 for v1.1. The design diagrams use UML notation covered in the CS4433 lectures and, where needed, OMG UML 2.5.1. The Entity-Relationship Diagrams use Crow's Foot notation.

3.1 Architecture

3.1.1 Architectural style and application type

The LMS is a transaction-based information system. Its desktop application accepts role-specific requests to read or modify enrolments, assignment submissions and assessment records. A layered information-system architecture is selected, adapting the structure presented in the CS4433 Architectural Design lecture (slides 18-19 and 35-39). All components execute on one school computer, with local SQLite persistence and no Internet dependency during normal use. The second laptop is a packaging-test target, not an LMS client.

3.1.2 Components and their responsibilities

Logical layer	Components	Responsibility
Presentation	Desktop GUI	Login and role screens; gathers input and shows results; contains no SQL or late-penalty calculation.
Application/security	AuthenticationService, AcademicManagementService, SubmissionManagementService, MarkingModerationService	Checks identity, role and course/department scope and coordinates each use case.
Business rules	User and academic domain objects; DeadlineExtension; ILatePenaltyPolicy and its implementation	Calculates effective deadlines, permitted resubmissions, penalties and moderated/released status.
Persistence	Repository contracts, SQLiteRecordRepository, ISubmissionContentStore and LocalContentStore, IPasswordHasher and LibraryPasswordHasher	Reads/writes SQL using prepared statements and transactions; manages staged attachments through a separate contract.

3.1.3 Component diagram and key interfaces

Figure 3.1 shows the main application components and their required and provided interfaces. The Desktop GUI depends on application services, while those services use repository and storage interfaces. The persistence adapters manage SQLite records and attachments without placing database operations in the GUI.

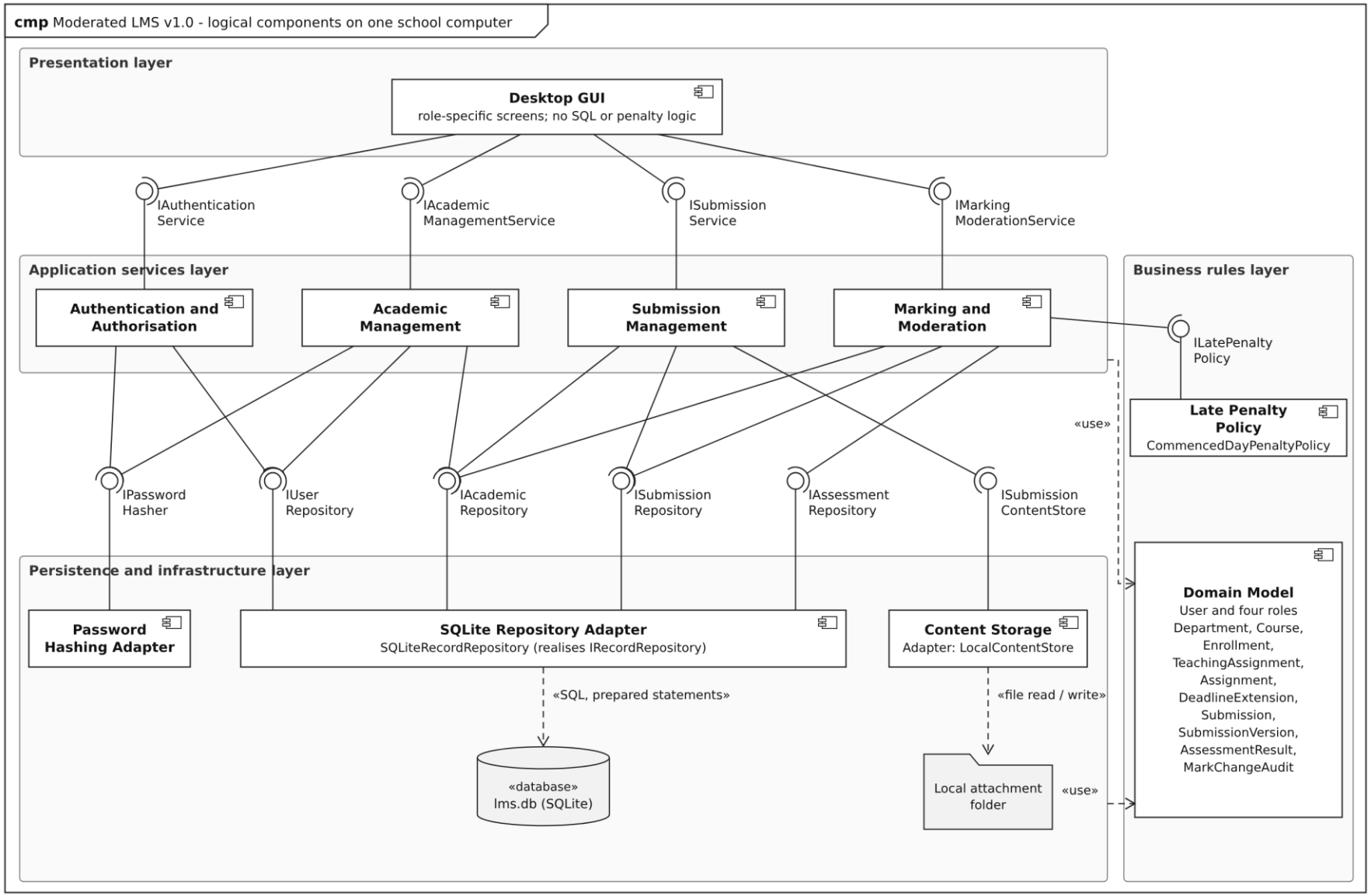


Figure 3.1 - UML Component Diagram. Logical components of the single-computer Moderated LMS, with provided and required interfaces.

3.1.4 Architectural rationale and rejected alternative

Architectural choice	Advantages	Limitations / decision
Selected: Layered information system	Consistent authorisation rules; separate GUI, assessment policy and storage; business rules independently testable.	Interfaces add complexity and may have overhead; keep boundaries focused.
Rejected: Distributed client-server	Could share records over several machines.	Adds a server, network reliability and administration that v1.0 does not require; clashes with selected one-computer deployment.
Alternative: MVC	Useful separation of GUI data display and interaction.	Can structure the desktop interface internally, but does not by itself define the security, audit and persistence boundaries needed by the overall LMS.

The selected architecture decomposes the desktop application into logical components without requiring a distributed system. It separates the GUI, authorisation, business rules and persistence responsibilities. This supports offline operation (NFR-02), credential security (NFR-03), usability (NFR-04), controlled release of marks (BR-01) and mark-change auditing (FR-18). The GUI framework will be selected before implementation.

3.2 Detailed Class Design

3.2.1 Principal classes and derivation

The principal classes were identified from the nouns and entities in the SRS, required system behaviour, and the UC-01, UC-02 and UC-03 use cases. Figures 3.2a-3.2e present the class model in five views so that the attributes, methods, interfaces, relationships and multiplicities remain readable. Classes shown with only their names are detailed in another view.

SRS basis	Classes	Why they are needed
FR-01-FR-02	User (abstract), Student, Lecturer, HeadOfDepartment, Administrator	Shared identity and polymorphic role-specific type.
FR-03-FR-10	Department, Course, Enrollment, TeachingAssignment, Assignment, DeadlineExtension	Academic structure, assignments, role allocations and individual extensions.
FR-11-FR-13	Submission, SubmissionVersion	One student-assignment submission record containing immutable attempts.
FR-14-FR-18	AssessmentResult, MarkChangeAudit	Mark, penalty, approval/release metadata and tracked revisions.
Architecture / testability	Four service interfaces and classes, Session and SubmissionContent data types, ILatePenaltyPolicy,	Coordinates use cases and separates policy from persistence.

SRS basis	Classes	Why they are needed
	IPasswordHasher, repository and content interfaces	

3.2.2 Object relationships, multiplicities and inheritance

Figure 3.2a shows the four subclasses of the abstract User class, Department-Course, course enrolments and lecturer allocations. Enrollment and TeachingAssignment are drawn as UML association classes on the many-to-many Student-Course and Lecturer-Course associations. An individual DeadlineExtension refers to exactly one assignment, one student and a granting lecturer. No whole-class extension is designed into v1.0 because that change belongs to the later CR1 maintenance release.

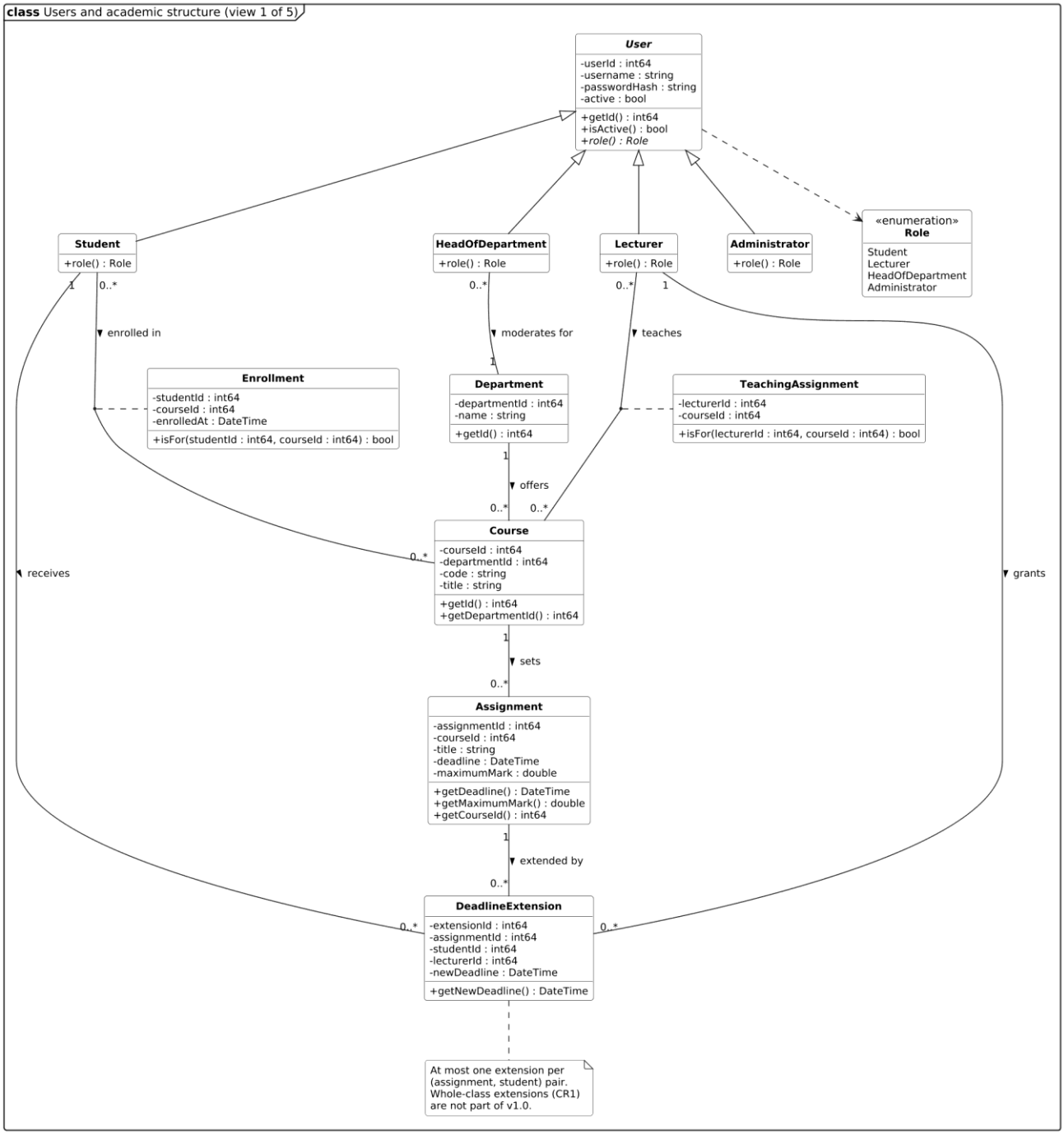


Figure 3.2a - Class Diagram view 1: abstract User hierarchy and academic domain relationships.

A Submission has one or more SubmissionVersion objects, and at most one current AssessmentResult; previous versions remain retained. Each AssessmentResult points to the exact version that was marked. A revised existing mark must generate a corresponding MarkChangeAudit row including the changed numeric field, previous value, new value, actor, time and reason. AssessmentResult has a directed association to the exact SubmissionVersion it marks, and associations record the marking lecturer, the moderating HoD and the user responsible for each audit event.

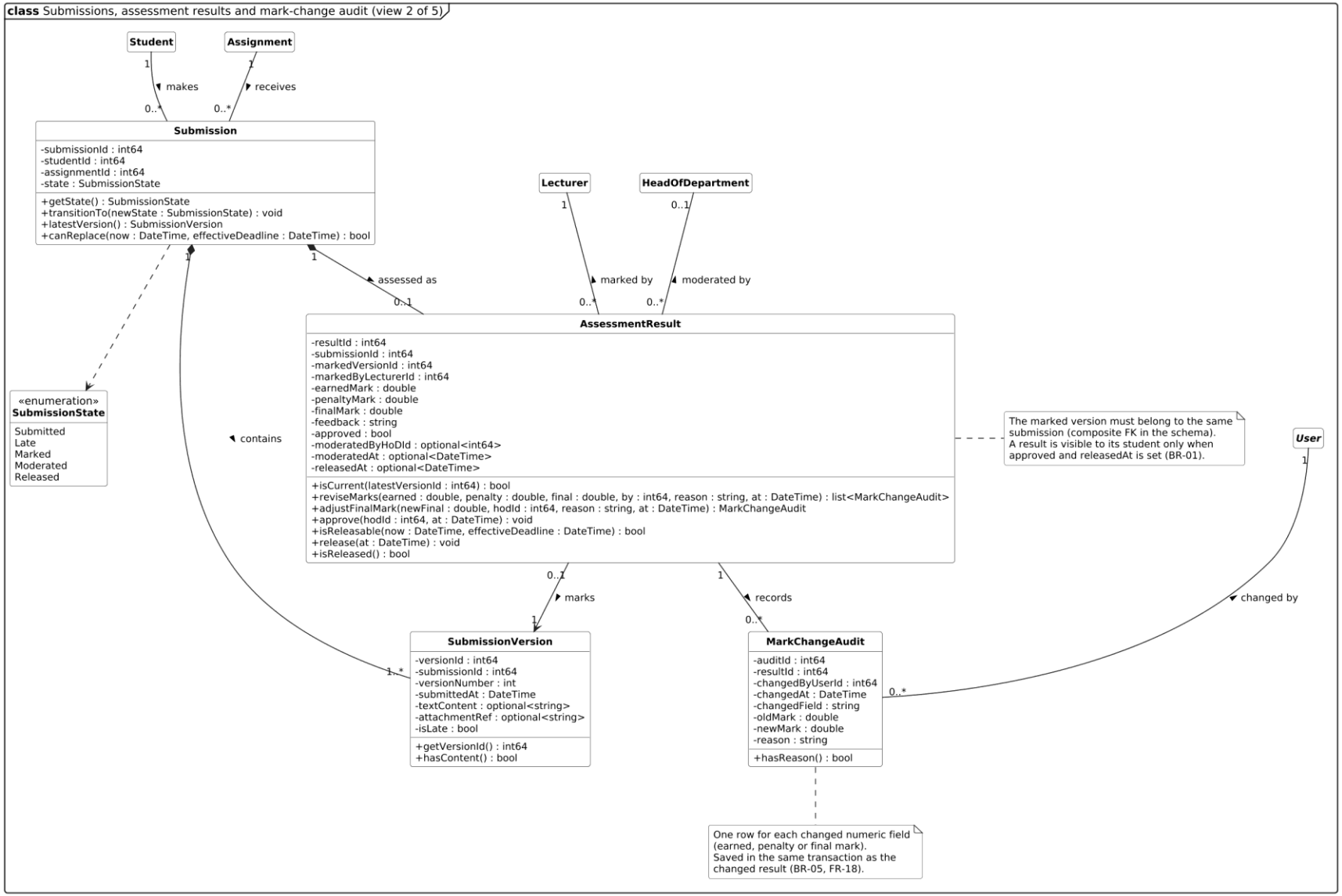


Figure 3.2b - Class Diagram view 2: submission versions, assessment result fields and mark-change audit.

3.2.3 Service classes and design patterns

Figure 3.2c lists the full operation signatures of the four application service interfaces used by the GUI. Figure 3.2d shows the design patterns as UML relationships: each service holds navigable associations only to the interfaces it needs, MarkingModerationService holds the penaltyPolicy reference of the Strategy pattern, and SQLiteRecordRepository realises the composite repository contract. Appendix B reproduces these interfaces at a readable size without changing the Class Diagram.

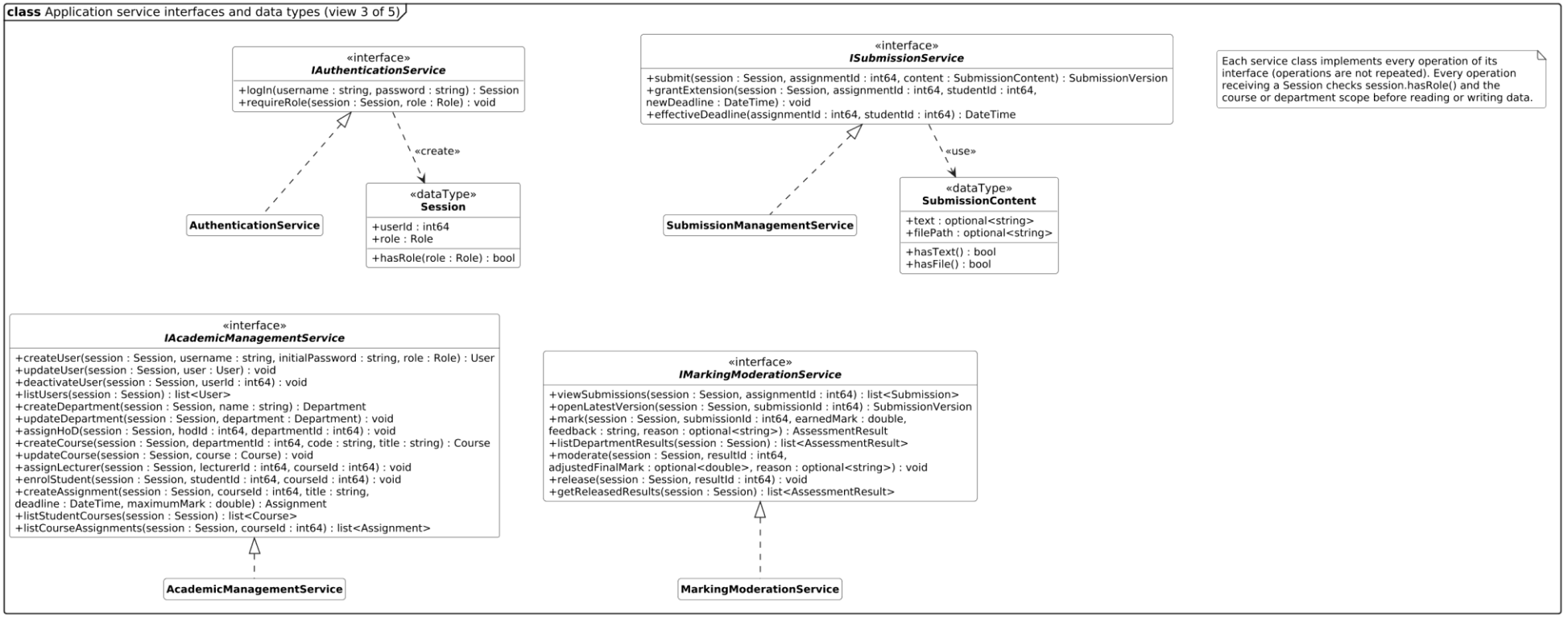


Figure 3.2c - Class Diagram view 3: application service interfaces, implementing service classes and data types.

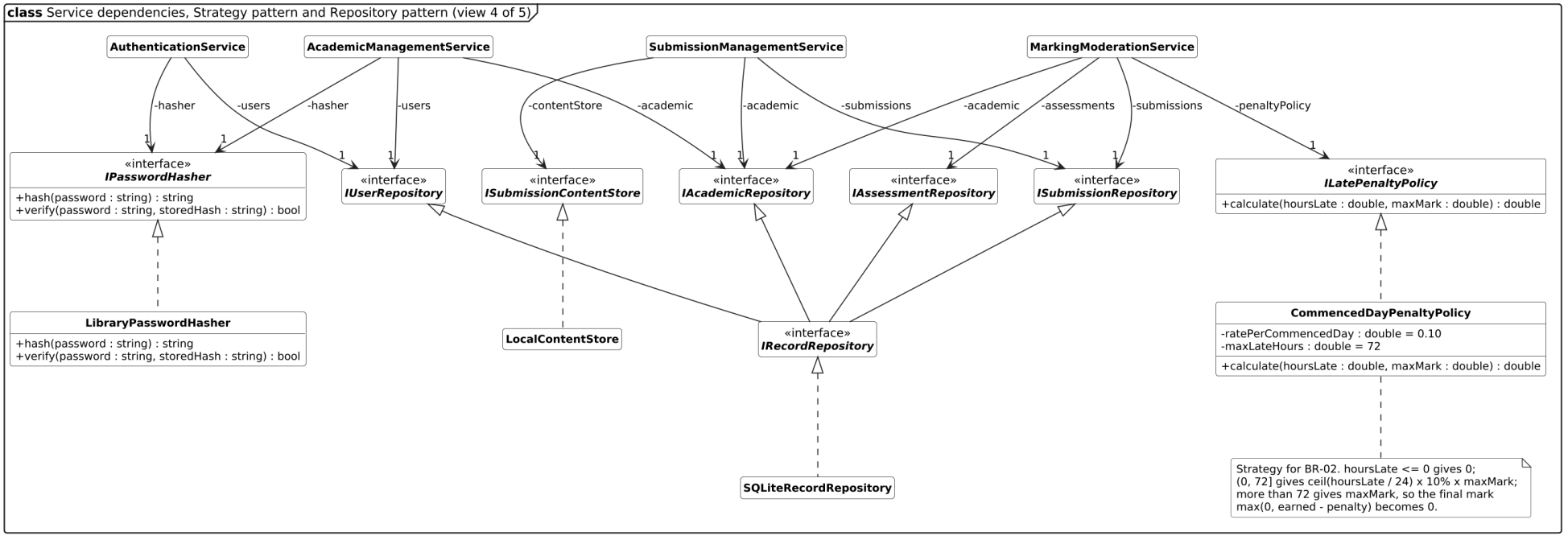


Figure 3.2d - Class Diagram view 4: service dependencies, Strategy pattern (late penalty) and Repository/DAO pattern.

Pattern (name)	Problem and solution	Trade-off / consequence
Strategy	ILatePenaltyPolicy defines calculate(hoursLate, maxMark). CommencedDayPenaltyPolicy implements Week 2 BR-02. MarkingModerationService holds a penaltyPolicy association to the interface (Figure 3.2d), so no penalty conditionals are placed in the GUI or the service.	Rules can be independently tested and substituted; interface adds one level of indirection.
Repository / DAO	Four focused contracts (IUserRepository, IAcademicRepository, ISubmissionRepository and IAssessmentRepository) are combined by IRecordRepository and realised by SQLiteRecordRepository with parameterised queries and one transaction per write operation (Figures 3.2d and 3.2e).	Services use test doubles without opening production SQLite; SQL mapping adds a persistence implementation layer.

3.2.4 Persistence interfaces

In Figure 3.2e, dashed realisation arrows point from implementation classes to interfaces. All service operations receive or derive an authenticated actor; IDs supplied by the GUI are not trusted as proof of permission. When data access fails, the service rolls back its transaction and returns a controlled error. Files use stage/commit/discard/remove operations so an unsuccessful upload does not appear as a completed submission and committed orphan files can be cleaned up after SQL failures.

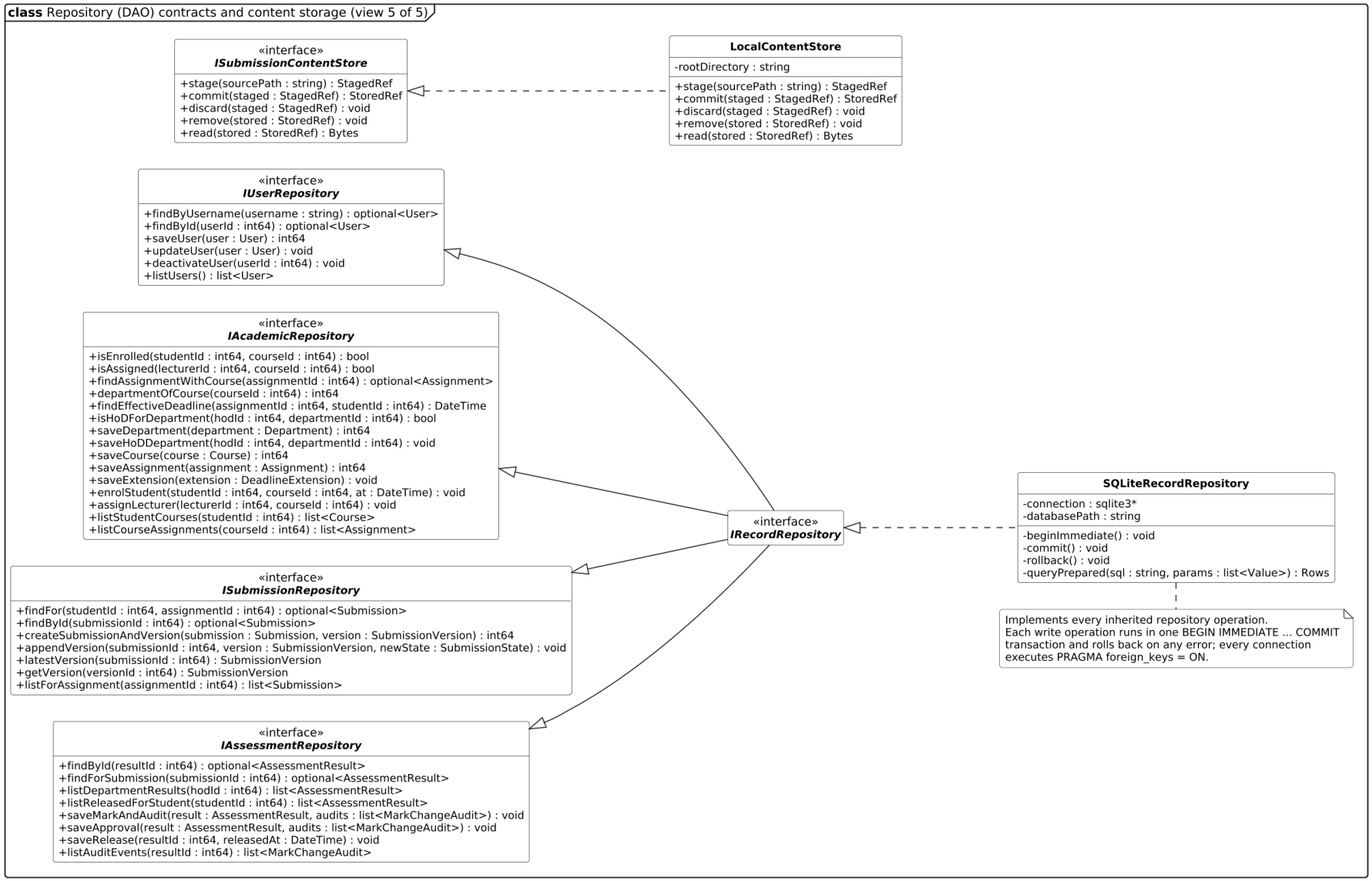


Figure 3.2e - Class Diagram view 5: repository (DAO) and submission-content interfaces, with implementing adapters.

3.2.5 SOLID principles applied

Principle	Concrete application
Single Responsibility (SRP)	Submission stores lifecycle identity, penalty policy calculates deductions and SQLite adapter persists records.
Open/Closed (OCP)	A changed late rule can be implemented using another ILatePenaltyPolicy without rewriting callers.
Liskov Substitution (LSP)	Student, Lecturer, HeadOfDepartment and Administrator each implement the abstract role() operation and can be used wherever a User is expected, for example as the result of IUserRepository.findByUsername().
Interface Segregation (ISP)	Each service depends only on the repository contracts it uses (Figure 3.2d); for example, SubmissionManagementService has no reference to IAssessmentRepository.
Dependency Inversion (DIP)	The GUI depends on the four service interfaces, and the services depend on ILatePenaltyPolicy, IPasswordHasher, ISubmissionContentStore and the repository interfaces rather than on concrete policy, hashing, file or database classes.

UML key: + public, - private (no protected members are needed in this design); an italic class or operation name is abstract; «interface», «enumeration» and «dataType» are standard keywords; solid line with hollow triangle = generalisation; dashed line with hollow triangle = interface realisation; filled diamond = composition; arrowhead on an association = navigability; dashed open arrow = dependency; dashed line to a class on an association = association class; 1, 0..1, 0..* and 1..* are multiplicities. Design-level types int64, DateTime, optional<T>, list<T>, StagedRef, StoredRef and Bytes map to C++17 types in implementation.

3.3 Behavioural Models

3.3.1 Submission state machine

The Submission state machine describes the lifecycle beginning with the first accepted version. A late first submission is accepted and recorded as Late. An existing submission may be replaced only on or before the student's effective deadline. Marks remain unreleased until approval and release by the Head of Department. Invalid replacements do not change the existing submission. Transitions use the event [guard] / action notation. A final state is not used because released records remain available for viewing and auditing.

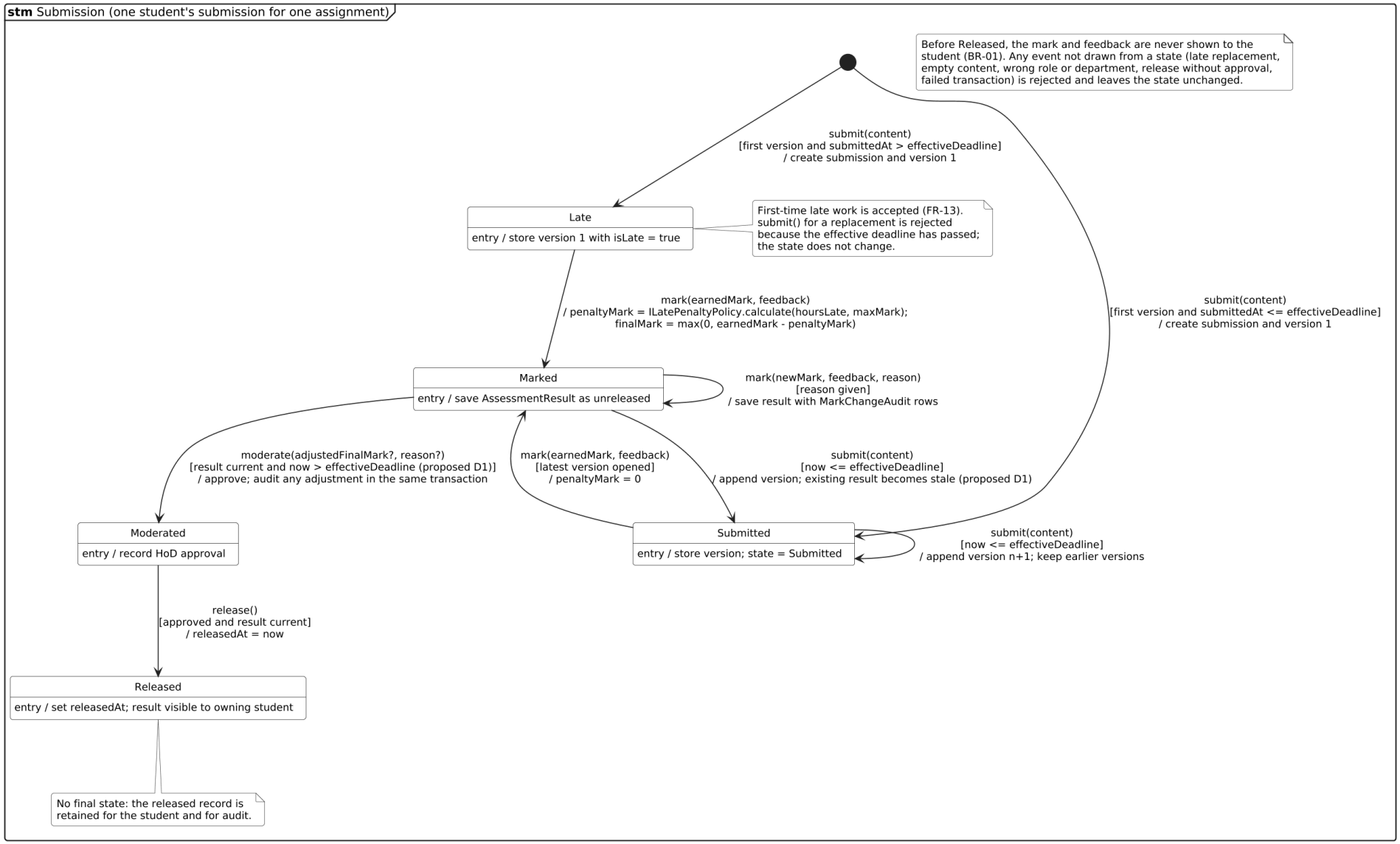


Figure 3.3a - Submission state machine with guards, actions and permitted re-marking after an on-time replacement.

3.3.2 End-to-end activity model

The Activity Diagram shows the submission, marking, moderation and release process using the Student, LMS, Lecturer and Head of Department swimlanes. The diagram is divided into two connected parts for readability. Only accepted submissions proceed to marking through connector A. A rejected resubmission ends that attempt without removing earlier accepted work. Decision nodes have guard conditions, and the process does not require parallel activities.

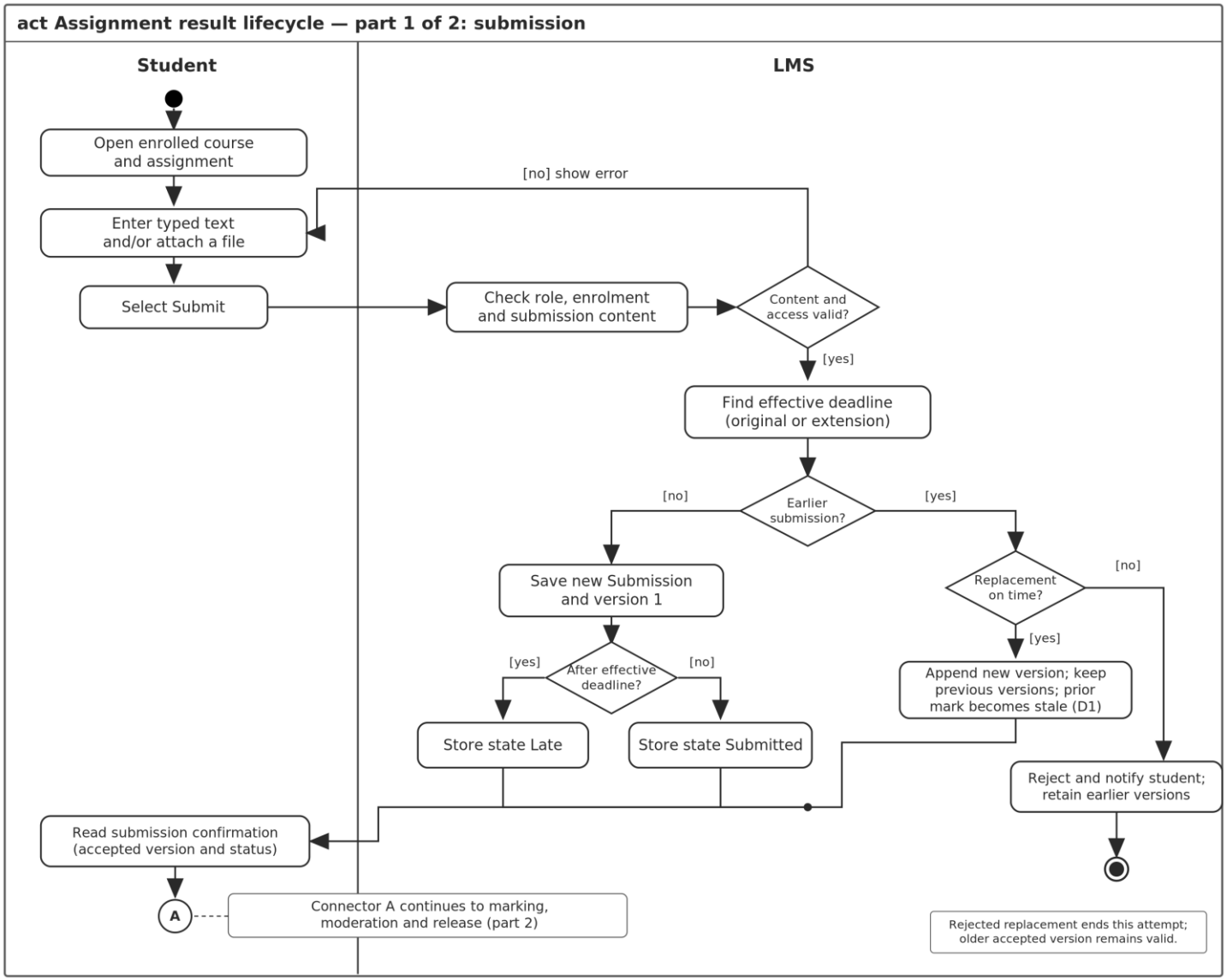


Figure 3.3b (part 1 of 2) - Activity Diagram with role swimlanes: submission and resubmission.

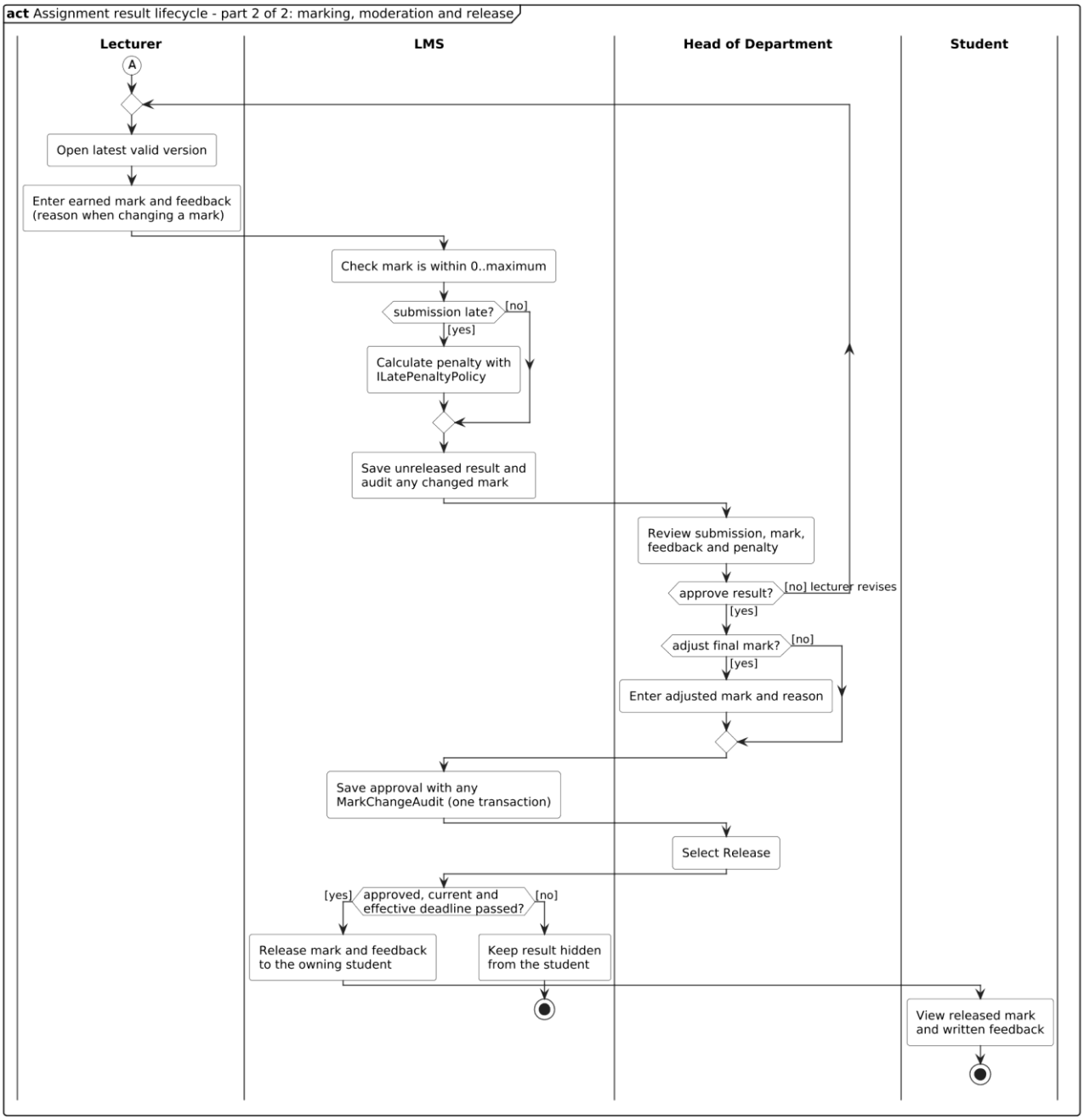


Figure 3.3b (part 2 of 2) - Activity Diagram with role swimlanes: marking, moderation and release.

3.3.3 UC-01 sequence: Submit assignment

SubmissionManagementService verifies the student's enrolment, assignment identity, effective deadline and content. First-time late work is accepted but is recorded as late. A replacement after the effective deadline is rejected. A file attachment is staged through ISubmissionContentStore; the repository write and file operation are coordinated so a failed operation is rolled back/compensated and success is only reported when required content and metadata are present. The attachment is staged and then committed before one atomic repository write; if that write fails, the committed file is removed. Part 2 is referenced from part 1 by a UML ref interaction use, and every message corresponds to an operation in Figures 3.2b-3.2e.

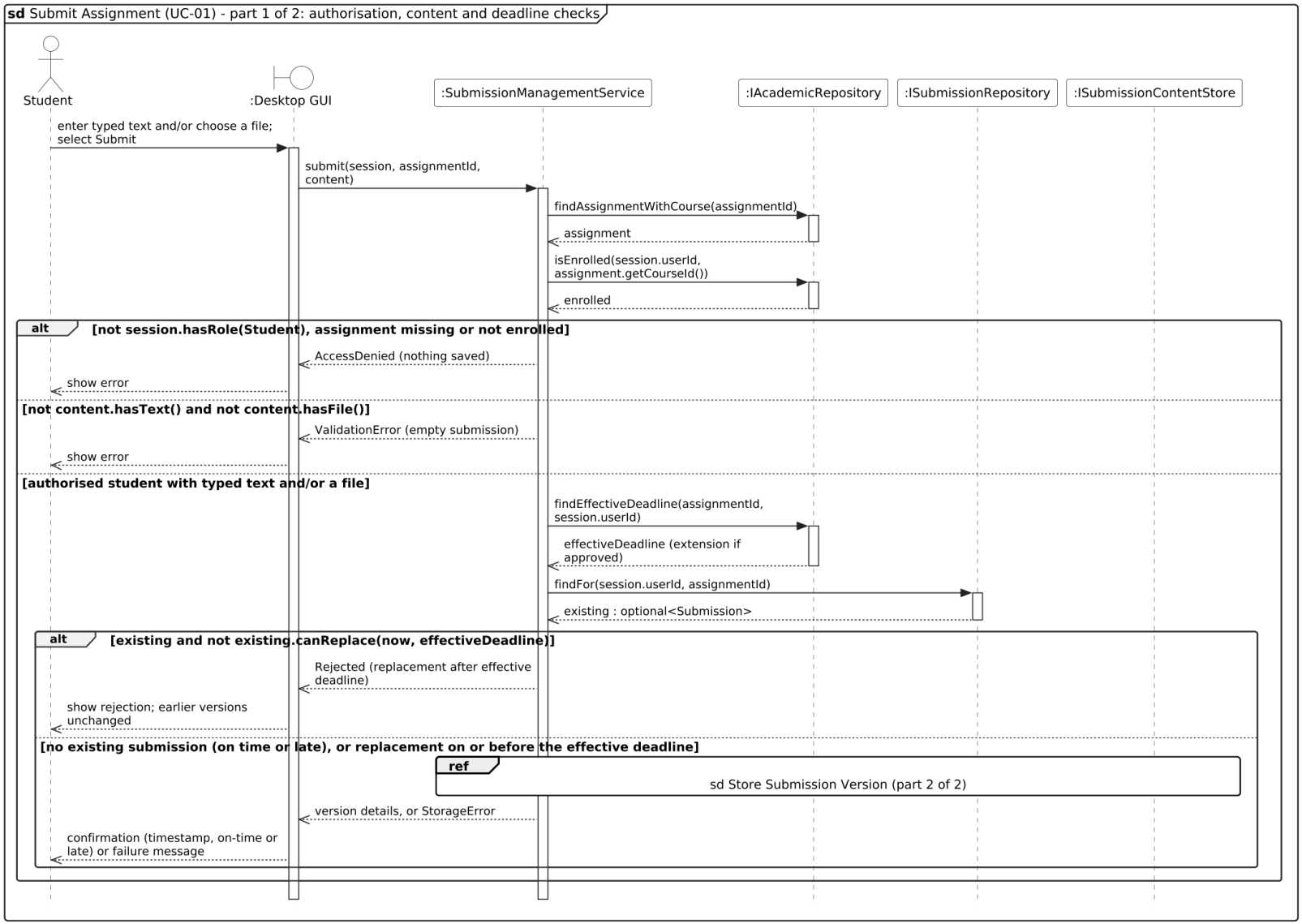


Figure 3.3c (part 1 of 2) - UC-01 sequence: authorisation, content, effective deadline and replacement checks.

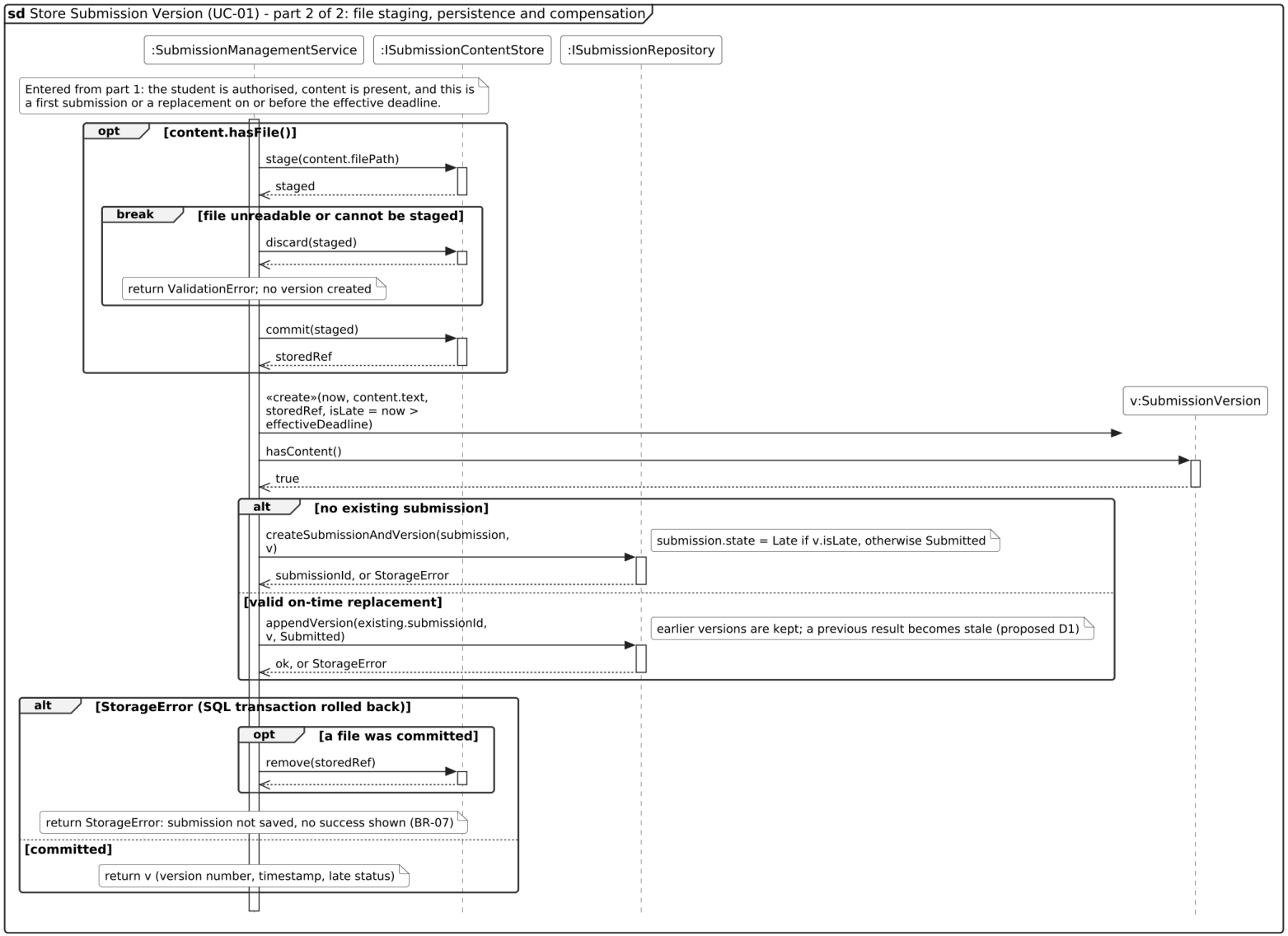


Figure 3.3c (part 2 of 2) - UC-01 sequence: attachment staging, version persistence and failure compensation.

3.3.4 UC-03 sequence: Moderate and release marks

MarkingModerationService verifies the HoD's department, retrieves the assessment and checks that the marked version is current. An adjustment requires a reason and is saved atomically with an audit event for each changed numeric mark. Moderation approval precedes release. Only the student who owns an approved, released result can retrieve it; a denied or failed operation leaves the previous state unchanged. Figure 3.3d shows moderation in two parts and Figure 3.3e shows release and the student's read access.

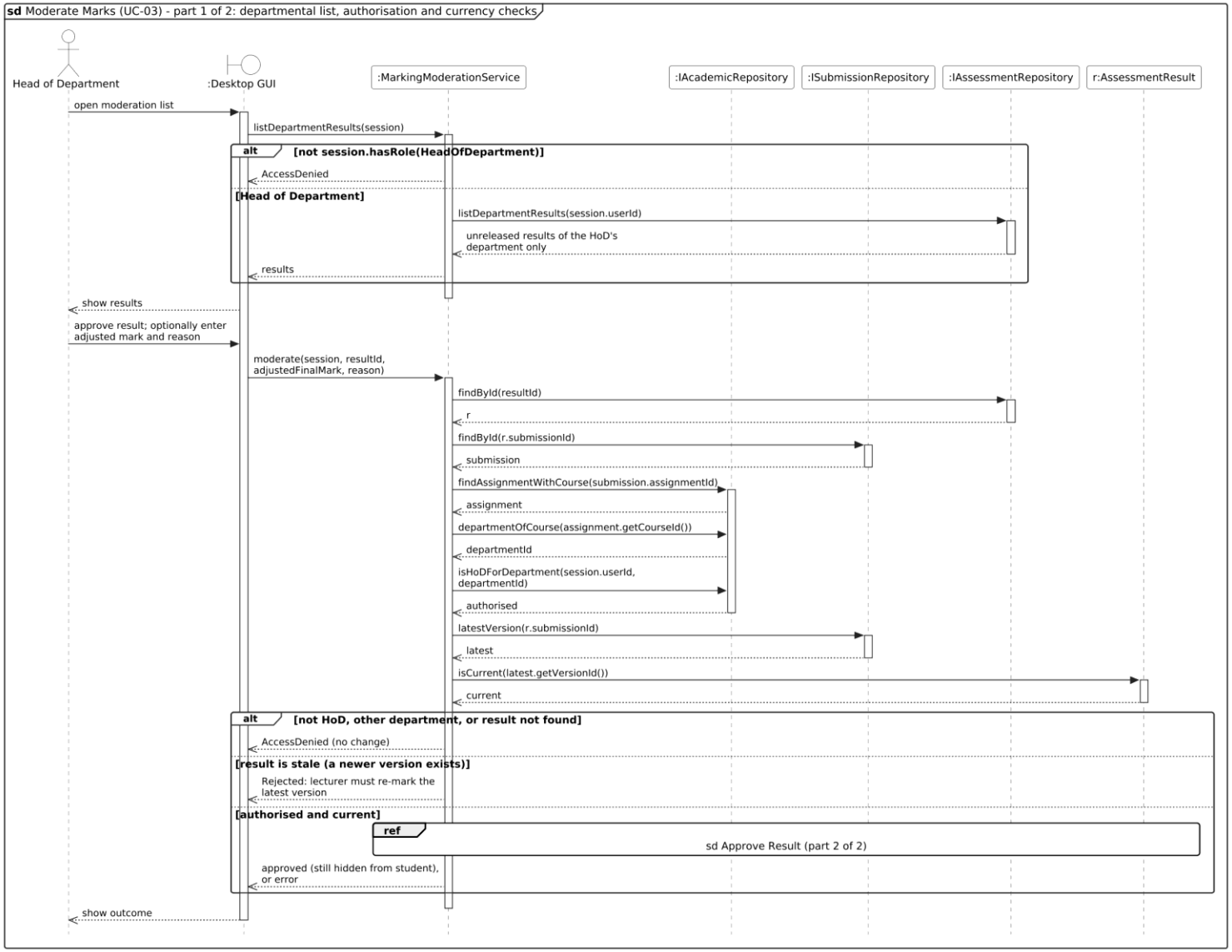


Figure 3.3d (part 1 of 2) - UC-03 sequence: departmental list, authorisation and currency checks.

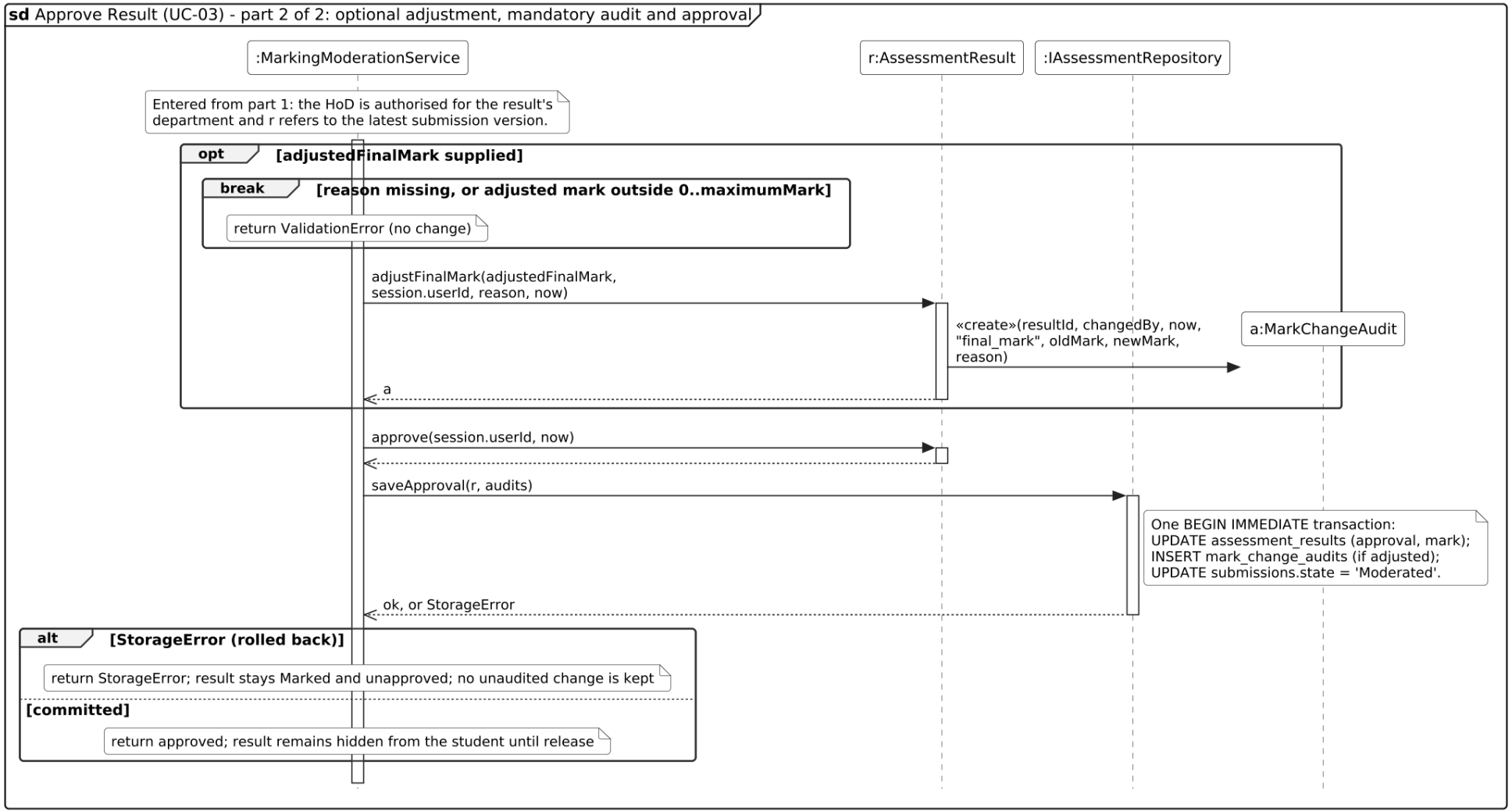


Figure 3.3d (part 2 of 2) - UC-03 sequence: optional adjustment, mandatory audit record and approval.

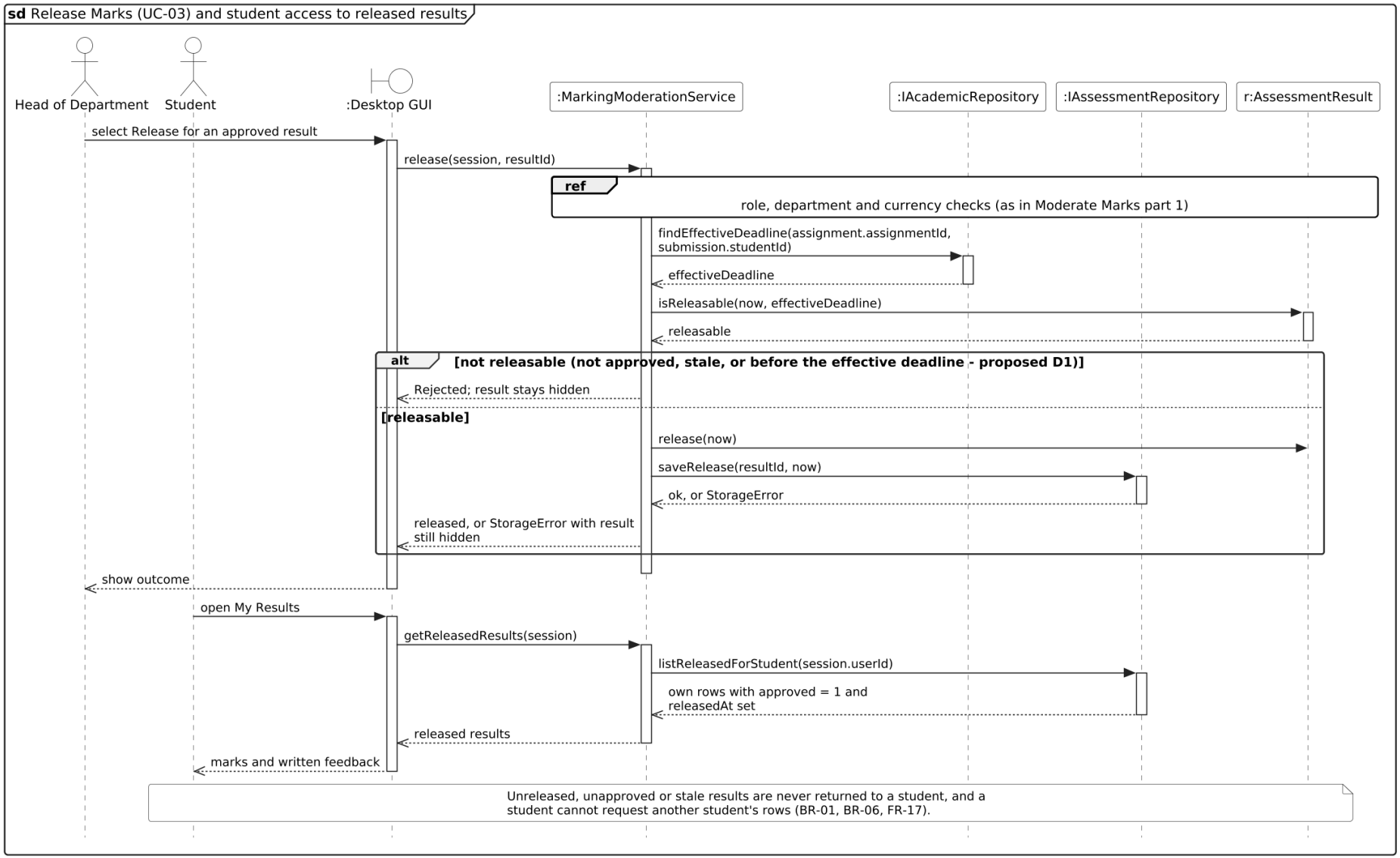


Figure 3.3e - UC-03 sequence: controlled release and protection of unreleased results.

3.3.5 Exact timing and exception rules

Condition	Required design behaviour
At deadline	A resubmission stamped exactly at the effective deadline is permitted (BR-04).
Late (0,24] hours	One commenced late day; 10% of maximum deducted from earned mark.
Late (24,48] and (48,72] hours	Two and three late days respectively; 20% and 30% of maximum deducted.
More than 72 hours late	First submission is retained; resulting final mark is zero (FR-13).
Approved individual extension	Its new deadline replaces the original in the relevant student's checks.
On-time replacement after early mark	Proposed D1: prior mark is stale when markedVersionId differs from latest version; lecturer must re-mark newest version, preserving revisions and audit history.
Release before effective deadline	Proposed D1: not permitted; avoids releasing marks while FR-12 still permits replacement.
SQL or file error	Rollback database transaction, discard staged content/compensate and report failure (BR-07).
Wrong actor/course/department	Reject before any record change (FR-02, BR-06).

The treatment of early marking and permitted resubmissions is defined in the design assumptions in Section 6. The late-penalty intervals follow BR-02 of the submitted SRS.

3.4 Database Design

3.4.1 Database and relationship model

SQLite is used for the single-computer offline deployment. The schema has 12 tables. Figure 3.4a shows the academic structure and Figure 3.4b shows submissions, assessment and audit events. The figures are two related views of the same database; repeated tables are shared references, not separate entities. PK is primary key and FK is foreign key. Application validation complements SQL PK/FK/UNIQUE/CHECK constraints. The ERD uses Crow's Foot notation: || is exactly one, |o is zero or one, o{ is zero or many and |{ is one or many; * marks a NOT NULL column and UQ1 marks columns that form one composite UNIQUE constraint.

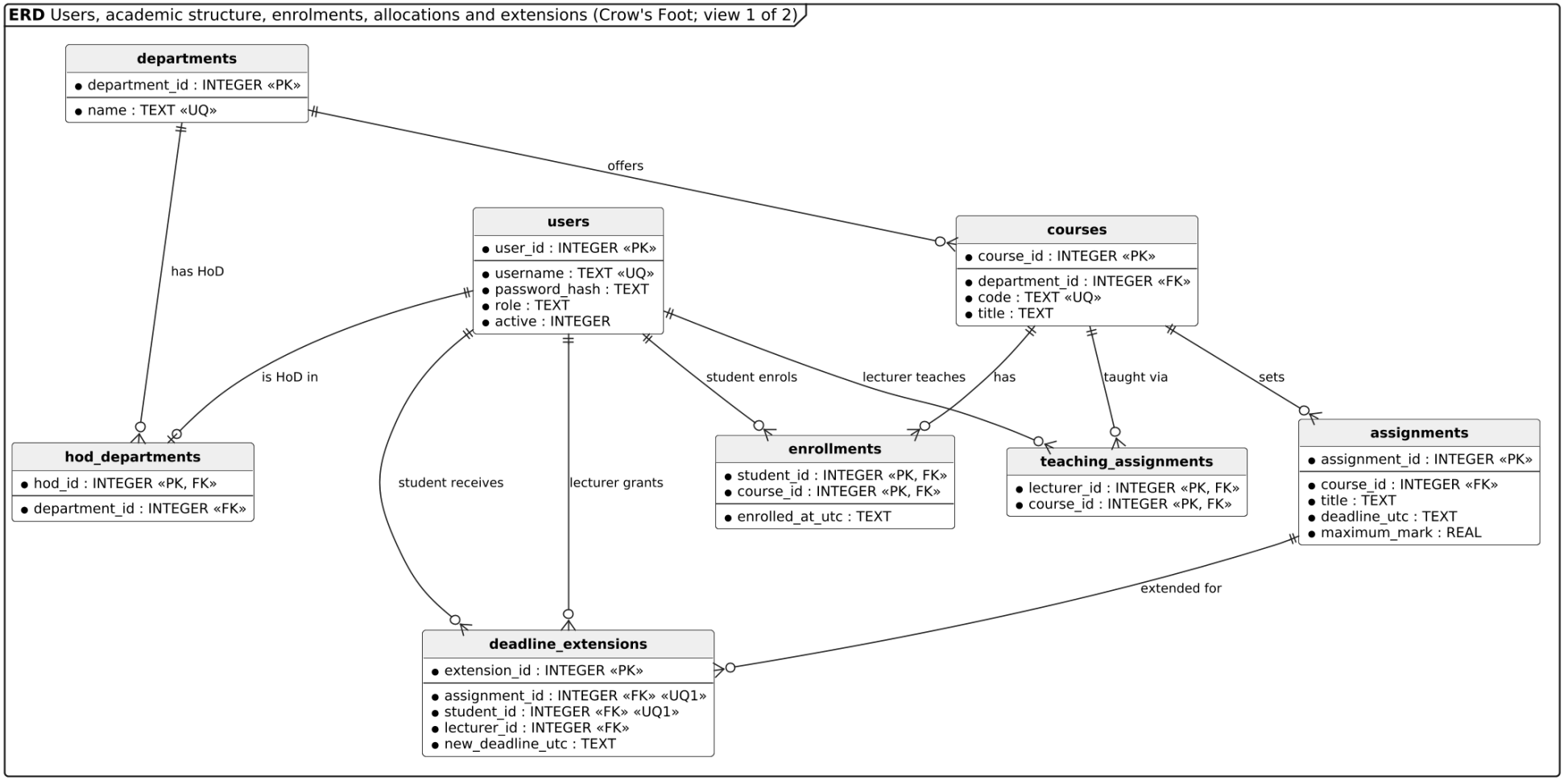


Figure 3.4a - ERD view 1: users, academic structure, enrolments, allocations and extensions.

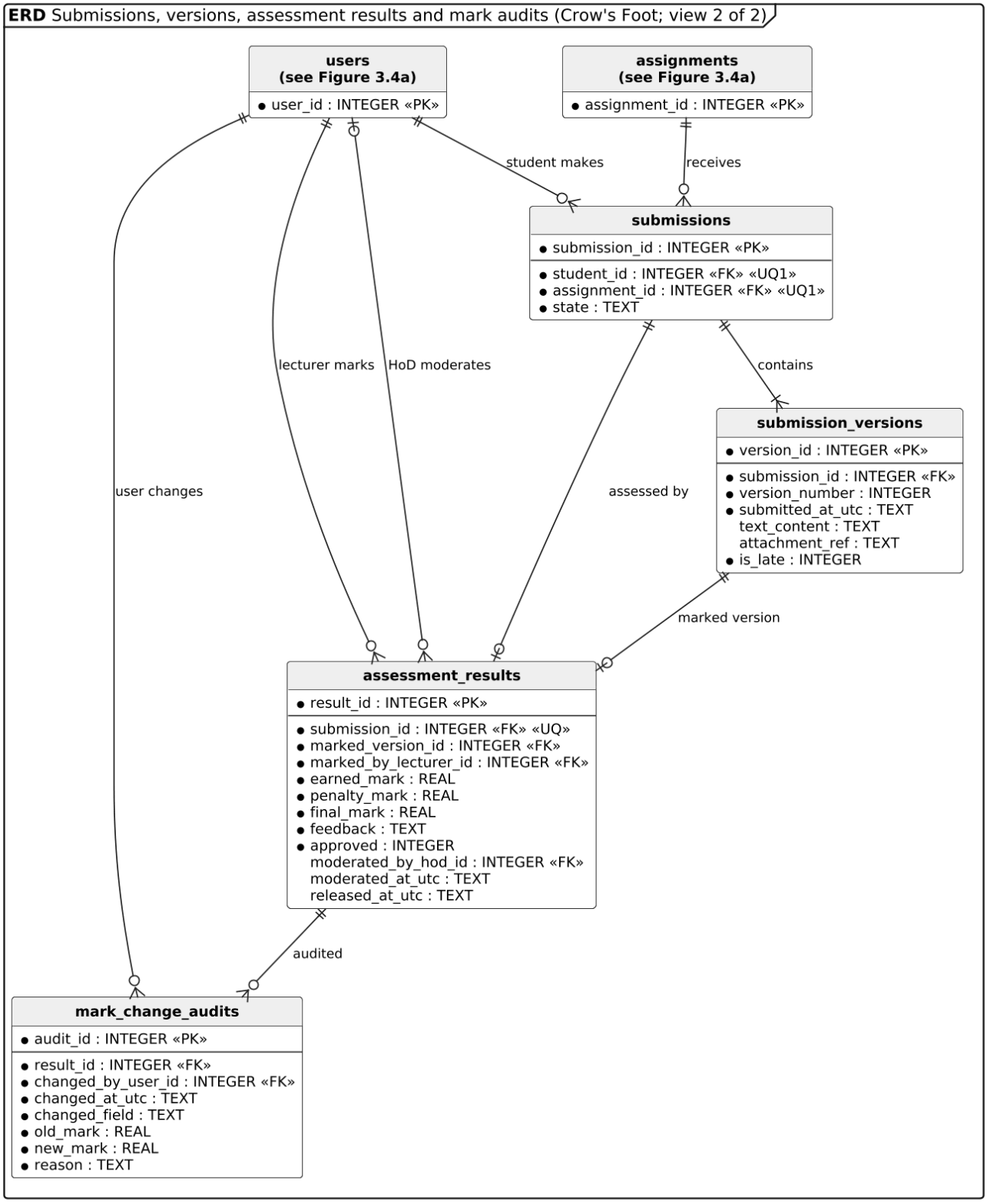


Figure 3.4b - ERD view 2: submissions, immutable versions, assessment results and mark audits.

3.4.2 Normalisation to Third Normal Form

Normal form	Evidence in this design
1NF	Every table column holds a scalar value; repeated submission attempts are separate rows in submission_versions, not repeated fields in submissions.
2NF	Composite-key tables

Normal form	Evidence in this design
	enrollments(student_id,course_id) and teaching_assignments(lecturer_id,course_id) do not store facts depending on only one component; the enrollment timestamp refers to the whole association.
3NF	Departments, courses and users hold their own names/titles/credentials. Course title is not duplicated in assignments; the user role is not duplicated in enrolments; audit changes are stored in a separate event table. In assessment_results, marked_version_id determines submission_id, but submission_id is itself a candidate key (UNIQUE NOT NULL), so the dependency ends in a key attribute and 3NF is still satisfied.

The final_mark column is retained as the authoritative awarded assessment value after moderation, not merely a calculated alias: it can differ from earned_mark minus penalty_mark when the HoD makes an approved adjustment under BR-08. The state and is_late fields are operational facts and must be kept coherent by the service layer. No unsupported normalisation claims are made for hypothetical future extensions.

3.4.3 Data dictionary (SQLite DDL aligned)

The data dictionary specifies the database columns, SQLite types and key constraints. INTEGER is used for identifiers and Boolean values, TEXT for names, timestamps and references, and REAL for marks. Timestamps are stored in UTC.

assessment_results

Column	SQLite type	Key / requirement
result_id	INTEGER	PK (rowid), never NULL
submission_id	INTEGER	FK,NOT NULL,UNIQUE
marked_version_id	INTEGER	FK with submission_id,NOT NULL
marked_by_lecturer_id	INTEGER	FK,NOT NULL
earned_mark	REAL	NOT NULL
penalty_mark	REAL	NOT NULL
final_mark	REAL	NOT NULL
feedback	TEXT	NOT NULL
approved	INTEGER	NOT NULL,default 0
moderated_by_hod_id	INTEGER	FK,nullable
moderated_at_utc	TEXT	nullable
released_at_utc	TEXT	nullable

assignments

Column	SQLite type	Key / requirement
assignment_id	INTEGER	PK (rowid), never NULL
course_id	INTEGER	FK,NOT NULL

Column	SQLite type	Key / requirement
title	TEXT	NOT NULL
deadline_utc	TEXT	NOT NULL
maximum_mark	REAL	NOT NULL,CHECK > 0

courses

Column	SQLite type	Key / requirement
course_id	INTEGER	PK (rowid), never NULL
department_id	INTEGER	FK,NOT NULL
code	TEXT	NOT NULL,UNIQUE
title	TEXT	NOT NULL

deadline_extensions

Column	SQLite type	Key / requirement
extension_id	INTEGER	PK (rowid), never NULL
assignment_id	INTEGER	FK,NOT NULL,UNIQUE pair
student_id	INTEGER	FK,NOT NULL,UNIQUE pair
lecturer_id	INTEGER	FK,NOT NULL
new_deadline_utc	TEXT	NOT NULL

departments

Column	SQLite type	Key / requirement
department_id	INTEGER	PK (rowid), never NULL
name	TEXT	NOT NULL,UNIQUE

enrollments

Column	SQLite type	Key / requirement
student_id	INTEGER	PK,FK,NOT NULL
course_id	INTEGER	PK,FK,NOT NULL
enrolled_at_utc	TEXT	NOT NULL

hod_departments

Column	SQLite type	Key / requirement
hod_id	INTEGER	PK,FK,never NULL
department_id	INTEGER	FK,NOT NULL

mark_change_audits

Column	SQLite type	Key / requirement
audit_id	INTEGER	PK (rowid), never NULL
result_id	INTEGER	FK,NOT NULL
changed_by_user_id	INTEGER	FK,NOT NULL

Column	SQLite type	Key / requirement
changed_at_utc	TEXT	NOT NULL
changed_field	TEXT	NOT NULL,CHECK in 3 fields
old_mark	REAL	NOT NULL
new_mark	REAL	NOT NULL
reason	TEXT	NOT NULL

submission_versions

Column	SQLite type	Key / requirement
version_id	INTEGER	PK (rowid), never NULL
submission_id	INTEGER	FK,NOT NULL
version_number	INTEGER	NOT NULL,UNIQUE per submission
submitted_at_utc	TEXT	NOT NULL
text_content	TEXT	nullable; text or file required
attachment_ref	TEXT	nullable; text or file required
is_late	INTEGER	NOT NULL

submissions

Column	SQLite type	Key / requirement
submission_id	INTEGER	PK (rowid), never NULL
student_id	INTEGER	FK,NOT NULL,UNIQUE pair
assignment_id	INTEGER	FK,NOT NULL,UNIQUE pair
state	TEXT	NOT NULL,CHECK in 5 states

teaching_assignments

Column	SQLite type	Key / requirement
lecturer_id	INTEGER	PK,FK,NOT NULL
course_id	INTEGER	PK,FK,NOT NULL

users

Column	SQLite type	Key / requirement
user_id	INTEGER	PK (rowid), never NULL
username	TEXT	NOT NULL,UNIQUE (NOCASE)
password_hash	TEXT	NOT NULL
role	TEXT	NOT NULL,CHECK in 4 roles
active	INTEGER	NOT NULL,default 1

Total: 12 application tables and 59 columns. The schema source file is included separately and reproduced in Appendix A.

3.4.4 Class-to-table mapping

Class / model type	Database representation	Reason for difference
User, four concrete role subclasses	users.role + users common identity fields	Single-table role discriminator preserves one login identity; polymorphic subclasses are reconstructed in C++.
Department / HoD assignment	departments + hod_departments	A separate HoD-department link stores the HoD organisational assignment.
Course, Enrollment, TeachingAssignment	courses, enrollments, teaching_assignments	Associative classes resolve many-to-many links through PK/FK pairs.
Assignment, DeadlineExtension	assignments, deadline_extensions	An extension belongs to one student-assignment pair; CR1 class extension is not present in v1.0.
Submission, SubmissionVersion	submissions, submission_versions	Separate tables preserve multiple immutable attempts while keeping one logical submission.
AssessmentResult, MarkChangeAudit	assessment_results, mark_change_audits	Audit stores multiple event rows instead of a repeated group of fields in results.
Service interfaces and classes, Session, SubmissionContent, ILatePenaltyPolicy, IPasswordHasher, repository interfaces	No domain tables	They contain transient behaviour / abstraction; SQLiteRecordRepository implements persistence rather than representing domain records.
LocalContentStore	submission_versions.attachment_ref + managed local file	Attachments are stored outside SQL behind an interface; physical location/limits require Gate 2 confirmation.

3.4.5 Database constraints and application-managed invariants

The DDL includes PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK and indexes. PRAGMA foreign_keys=ON is required for every SQLite connection. Because ordinary CHECK expressions cannot enforce cross-table school rules, authorised services must verify role, enrolment, lecturer allocation, HoD department, effective deadline, latest version and mark maximum before writing. A direct SQL UPDATE is not proof of compliance; production writes must use the repository service API.

Operation	Integrity rule / transaction responsibility
Create first submission	Insert submissions and first submission_versions row together; do not leave an empty submission aggregate.
Append version	Check effective deadline; preserve older versions; optionally make prior provisional assessment stale by version mismatch under

Operation	Integrity rule / transaction responsibility
	proposed D1.
Mark or moderate an existing value	Use BEGIN IMMEDIATE; UPDATE result and INSERT every associated mark_change_audits event; COMMIT together or ROLLBACK.
Moderate / release	Confirm authenticated department scope, approval, current marked version and proposed D1 time guard; do not allow student visibility until release.
Store attachment	Stage the file and check it can be read; commit the file to the managed folder; then run one atomic repository write (submission and version rows). If that write fails, roll back and remove the committed file. Crash recovery and orphan cleanup must be implemented and tested.

3.4.6 Schema verification

The SQL schema was executed using an in-memory SQLite database. The schema created 12 tables containing 59 columns. The foreign-key and database integrity checks passed. Constraint checks rejected invalid rows, including duplicate submissions, missing submission content, invalid states, incomplete release approval and audit records without a reason. Additional system and integration testing will be performed during implementation and verification.

5. Requirements-to-Design Traceability

The following table maps the functional requirements in the Week 2 SRS to the relevant design elements and database tables.

FR	Design model / operation	Persistence
FR-01	IAuthenticationService.logIn(); User.role()	users
FR-02	Session.hasRole() and scope checks in every service	users, scoped associations
FR-03	createUser(), updateUser(), deactivateUser(), listUsers()	users
FR-04	createDepartment(), updateDepartment(), assignHoD()	departments
FR-05	createCourse(), updateCourse()	courses
FR-06	assignLecturer()	teaching_assignments
FR-07	enrolStudent()	enrollments
FR-08	createAssignment()	assignments
FR-09	grantExtension(); findEffectiveDeadline()	deadline_extensions
FR-10	listStudentCourses(), listCourseAssignments()	enrollments, courses, assignments
FR-11	submit(); Figure 3.3c	submissions, submission_versions
FR-12	Submission.canReplace();	submission_versions

FR	Design model / operation	Persistence
	appendVersion(); Figure 3.3a	
FR-13	ILatePenaltyPolicy / CommencedDayPenaltyPolicy; Figure 3.3b	submission_versions, assessment_results
FR-14	viewSubmissions(), openLatestVersion(), mark()	assessment_results, submission_versions
FR-15	listDepartmentResults(), moderate(); Figure 3.3d	hod_departments, assessment_results
FR-16	release(); AssessmentResult.isReleasable(); Figure 3.3e	assessment_results
FR-17	getReleasedResults(); listReleasedForStudent()	assessment_results, submissions
FR-18	MarkChangeAudit; saveMarkAndAudit(), saveApproval()	mark_change_audits

6. Design Assumptions and Gate 2 Review

The following design assumptions are recorded for confirmation at Gate 2. They will be used consistently in implementation once accepted.

Design item	Selected design	Confirmation needed
Early marking and resubmission	A valid resubmission before the effective deadline makes earlier provisional marking outdated. The newest version must be marked before release. Moderation and release take place after the effective deadline.	Confirm the early-marking policy.
Late-penalty intervals	(0,24], (24,48] and (48,72] hours after the effective deadline follow BR-02. Submissions over 72 hours late receive zero.	Confirm the application of commenced-day penalties.
Submission attachments	Typed text, an attached file, or both will be accepted. PDF, DOCX and TXT with a proposed 10 MB limit will be stored locally.	Confirm supported formats, size limit and backup arrangement.
Audit record detail	Each change to an existing earned, penalty or final mark will have its own audit event showing the old value, new value, actor, time and reason.	Confirm the recorded fields.
Mark precision	Marks will use SQLite REAL and C++ double, with results rounded to two decimal places when stored.	Confirm the required rounding convention.

Design item	Selected design	Confirmation needed
HoD assignment	Each HoD will be linked to one department in v1.0. A department may have multiple HoDs.	Confirm department allocation rules.
Release and testing	Results will be released individually. The performance scenario contains 800 student accounts.	Confirm available hardware and test arrangements.
GUI framework	The GUI will be implemented using a desktop framework compatible with C++17 and offline deployment.	Select and document the toolkit before implementation.

The design will be reviewed at Gate 2 before implementation begins. The review will confirm the listed assumptions, the system interfaces, the database transaction boundaries and consistency between the diagrams.

References

National University of Lesotho, Department of Mathematics and Computer Science. (2026). CS4433 Software Engineering - Individual Assignment.

CS4433 lecture slides. Architectural Design, slides 3-6, 15, 18-19 and 35-39.

CS4433 lecture slides. Software Design and Implementation, slides 5-11 and 19-23.

CS4433 lecture slides. System Modelling, activity, use case, class, sequence and state diagram sections.

CS4433 lecture slides. Unified Modelling Language (use case, class, activity, sequence and state notation).

CS4433. UML in Software Engineering: Study Notes (sections 4-9 and common modelling mistakes).

CS4433 Week 7 notes. Design and Implementation, sections 2-3 on object-oriented design and patterns.

CS4433. Week 2 Submitted Requirements Engineering Document (FR-01-FR-18, NFR-01-NFR-06 and BR-01-BR-08).

Object Management Group. (2017). OMG Unified Modeling Language (UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Appendix A - SQLite Schema

This appendix contains the SQLite table definitions, keys, constraints and indexes used in the database design.

```
-- CS4433 Moderated LMS v1.0 - SQLite schema.
-- File formats, storage and early-marking behaviour are documented in Section 6.
-- BR-02 bounds are explicit: (0,24], (24,48], (48,72] hours after effective deadline.
-- Each SQLite connection MUST execute PRAGMA foreign_keys=ON.
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE users (
   user_id       INTEGER PRIMARY KEY,
   username      TEXT NOT NULL COLLATE NOCASE UNIQUE CHECK
(length(trim(username)) > 0),
   password_hash TEXT NOT NULL CHECK (length(trim(password_hash)) > 0),
   role          TEXT NOT NULL CHECK (role IN
('Student','Lecturer','HeadOfDepartment','Administrator')),
   active        INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1))
);

CREATE TABLE departments (
   department_id INTEGER PRIMARY KEY,
   name          TEXT NOT NULL COLLATE NOCASE UNIQUE CHECK (length(trim(name)) > 0)
);

-- Each Head of Department is associated with one department in v1.0.
CREATE TABLE hod_departments (
   hod_id       INTEGER PRIMARY KEY REFERENCES users(user_id) ON DELETE RESTRICT,
   department_id INTEGER NOT NULL REFERENCES departments(department_id) ON DELETE
RESTRICT
);

CREATE TABLE courses (
   course_id     INTEGER PRIMARY KEY,
   department_id INTEGER NOT NULL REFERENCES departments(department_id) ON DELETE
RESTRICT,
   code          TEXT NOT NULL COLLATE NOCASE UNIQUE CHECK (length(trim(code)) > 0),
   title         TEXT NOT NULL CHECK (length(trim(title)) > 0)
);

CREATE TABLE enrollments (
   student_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
   course_id  INTEGER NOT NULL REFERENCES courses(course_id) ON DELETE RESTRICT,
   enrolled_at_utc TEXT NOT NULL,
   PRIMARY KEY (student_id, course_id)
);

CREATE TABLE teaching_assignments (
   lecturer_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
   course_id   INTEGER NOT NULL REFERENCES courses(course_id) ON DELETE RESTRICT,
   PRIMARY KEY (lecturer_id, course_id)
);

CREATE TABLE assignments (
   assignment_id INTEGER PRIMARY KEY,
   course_id     INTEGER NOT NULL REFERENCES courses(course_id) ON DELETE
RESTRICT,
   title         TEXT NOT NULL CHECK (length(trim(title)) > 0),
   deadline_utc  TEXT NOT NULL,
```

```
    maximum_mark  REAL NOT NULL CHECK (maximum_mark > 0)
);

CREATE TABLE deadline_extensions (
    extension_id     INTEGER PRIMARY KEY,
    assignment_id    INTEGER NOT NULL REFERENCES assignments(assignment_id) ON
DELETE RESTRICT,
    student_id       INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    lecturer_id      INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    new_deadline_utc TEXT NOT NULL,
    UNIQUE (assignment_id, student_id)
);

CREATE TABLE submissions (
    submission_id INTEGER PRIMARY KEY,
    student_id    INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    assignment_id INTEGER NOT NULL REFERENCES assignments(assignment_id) ON DELETE
RESTRICT,
    state         TEXT NOT NULL CHECK (state IN
('Submitted','Late','Marked','Moderated','Released')),
    UNIQUE (student_id, assignment_id)
);

CREATE TABLE submission_versions (
    version_id     INTEGER PRIMARY KEY,
    submission_id  INTEGER NOT NULL REFERENCES submissions(submission_id) ON DELETE
RESTRICT,
    version_number INTEGER NOT NULL CHECK (version_number >= 1),
    submitted_at_utc TEXT NOT NULL,
    text_content   TEXT,
    attachment_ref TEXT,
    is_late        INTEGER NOT NULL CHECK (is_late IN (0,1)),
    UNIQUE (submission_id, version_number),
    UNIQUE (submission_id, version_id),
    CHECK (length(trim(COALESCE(text_content, ''))) > 0
       OR length(trim(COALESCE(attachment_ref, ''))) > 0)
);

CREATE TABLE assessment_results (
    result_id        INTEGER PRIMARY KEY,
    submission_id    INTEGER NOT NULL UNIQUE REFERENCES submissions(submission_id)
ON DELETE RESTRICT,
    marked_version_id INTEGER NOT NULL,
    marked_by_lecturer_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE
RESTRICT,
    earned_mark      REAL NOT NULL CHECK (earned_mark >= 0),
    penalty_mark     REAL NOT NULL CHECK (penalty_mark >= 0),
    final_mark       REAL NOT NULL CHECK (final_mark >= 0),
    feedback         TEXT NOT NULL CHECK (length(trim(feedback)) > 0),
```

```
  approved         INTEGER NOT NULL DEFAULT 0 CHECK (approved IN (0,1)),
  moderated_by_hod_id INTEGER REFERENCES users(user_id) ON DELETE RESTRICT,
  moderated_at_utc TEXT,
  released_at_utc  TEXT,
  FOREIGN KEY (submission_id, marked_version_id)
    REFERENCES submission_versions(submission_id, version_id) ON DELETE RESTRICT,
  CHECK (approved = 0 OR (moderated_by_hod_id IS NOT NULL AND moderated_at_utc IS
NOT NULL)),
  CHECK (released_at_utc IS NULL OR approved = 1)
);

CREATE TABLE mark_change_audits (
  audit_id       INTEGER PRIMARY KEY,
  result_id      INTEGER NOT NULL REFERENCES assessment_results(result_id) ON DELETE
RESTRICT,
  changed_by_user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE
RESTRICT,
  changed_at_utc TEXT NOT NULL,
  changed_field  TEXT NOT NULL CHECK (changed_field IN
('earned_mark','penalty_mark','final_mark')),
  old_mark       REAL NOT NULL CHECK (old_mark >= 0),
  new_mark       REAL NOT NULL CHECK (new_mark >= 0),
  reason         TEXT NOT NULL CHECK (length(trim(reason)) > 0)
);

CREATE INDEX idx_courses_department ON courses(department_id);
CREATE INDEX idx_enrollments_course ON enrollments(course_id);
CREATE INDEX idx_teaching_course ON teaching_assignments(course_id);
CREATE INDEX idx_assignments_course ON assignments(course_id);
CREATE INDEX idx_extensions_student ON deadline_extensions(student_id);
CREATE INDEX idx_submissions_assignment ON submissions(assignment_id);
CREATE INDEX idx_versions_submitted ON submission_versions(submission_id,
submitted_at_utc);
CREATE INDEX idx_results_moderator ON assessment_results(moderated_by_hod_id);
CREATE INDEX idx_audits_result ON mark_change_audits(result_id, changed_at_utc);

-- Example database read guard for released results (application must also authenticate and
authorise):
-- SELECT r.final_mark, r.feedback FROM assessment_results AS r
-- JOIN submissions AS s ON s.submission_id=r.submission_id
-- WHERE s.student_id=? AND r.approved=1 AND r.released_at_utc IS NOT NULL;
-- Required service transaction for mark revision:
-- BEGIN IMMEDIATE; UPDATE assessment_results ...; INSERT INTO mark_change_audits ...;
COMMIT;
-- On validation or storage error: ROLLBACK. Do not allow unaudited mark changes.

-- NOTE: Service-layer responsibilities not enforced by cross-table CHECKs:
-- 1. Authenticate actors and verify role, course, department, enrollment and HoD scope.
-- 2. Accept first-time late submissions; allow replacements only by the effective deadline.
```

```
-- 3. Provisional results are stale if their marked_version_id is not the newest version.
-- 4. Proposed Gate 2 policy: block moderation and release until effective deadline passes.
-- 5. Enforce earned_mark <= assignment maximum and agreed rounding policy.
-- 6. Persist every revised earned_mark/penalty_mark/final_mark together with one audit row per changed field
--    inside the same BEGIN IMMEDIATE/COMMIT transaction, else ROLLBACK.
-- 7. stage()/discard() attachment on failed save; reconcile local file and DB metadata.
```

Appendix B - Application Service Interface Signatures

Operations declared by the application service interfaces in Figure 3.2c.

Interface (from diagram)	Operations and signatures (unchanged)
«interface» IAuthenticationService	+ logIn(username : string, password : string) : Session + requireRole(session : Session, role : Role) : void
«interface» ISubmissionService	+ submit(session : Session, assignmentId : int64, content : SubmissionContent) : SubmissionVersion + grantExtension(session : Session, assignmentId : int64, studentId : int64, newDeadline : DateTime) : void + effectiveDeadline(assignmentId : int64, studentId : int64) : DateTime
«interface» IAcademicManagementService	+ createUser(session : Session, username : string, initialPassword : string, role : Role) : User + updateUser(session : Session, user : User) : void + deactivateUser(session : Session, userId : int64) : void + listUsers(session : Session) : list<User> + createDepartment(session : Session, name : string) : Department + updateDepartment(session : Session, department : Department) : void + assignHoD(session : Session, hodId : int64, departmentId : int64) : void + createCourse(session : Session, departmentId : int64, code : string, title : string) : Course + updateCourse(session : Session, course : Course) : void + assignLecturer(session : Session, lecturerId : int64, courseId : int64) : void + enrolStudent(session : Session, studentId : int64, courseId : int64) : void + createAssignment(session : Session, courseId : int64, title : string, deadline : DateTime, maximumMark : double) : Assignment + listStudentCourses(session : Session) : list<Course> + listCourseAssignments(session : Session, courseId : int64) : list<Assignment>

Interface (from diagram)	Operations and signatures (unchanged)
«interface» IMarkingModerationService	+ viewSubmissions(session : Session, assignmentId : int64) : list<Submission> + openLatestVersion(session : Session, submissionId : int64) : SubmissionVersion + mark(session : Session, submissionId : int64, earnedMark : double, feedback : string, reason : optional<string>) : AssessmentResult + listDepartmentResults(session : Session) : list<AssessmentResult> + moderate(session : Session, resultId : int64, adjustedFinalMark : optional<double>, reason : optional<string>) : void + release(session : Session, resultId : int64) : void + getReleasedResults(session : Session) : list<AssessmentResult>

Appendix C - Repository and Content-Storage Contracts

Repository and content storage operations from Figure 3.2e.

Interface (from diagram)	Operations and signatures (unchanged)
«interface» **IUserRepository**	+ findByUsername(username : string) : optional<User> + findById(userId : int64) : optional<User> + saveUser(user : User) : int64 + updateUser(user : User) : void + deactivateUser(userId : int64) : void + listUsers() : list<User>
«interface» **IAcademicRepository**	+ isEnrolled(studentId : int64, courseId : int64) : bool + isAssigned(lecturerId : int64, courseId : int64) : bool + findAssignmentWithCourse(assignmentId : int64) : optional<Assignment> + departmentOfCourse(courseId : int64) : int64 + findEffectiveDeadline(assignmentId : int64, studentId : int64) : DateTime + isHoDForDepartment(hodId : int64, departmentId : int64) : bool + saveDepartment(department : Department) : int64 + saveHoDDepartment(hodId : int64, departmentId : int64) : void + saveCourse(course : Course) : int64 + saveAssignment(assignment : Assignment) : int64 + saveExtension(extension : DeadlineExtension) : void + enrolStudent(studentId : int64, courseId : int64, at : DateTime) : void + assignLecturer(lecturerId : int64, courseId : int64) : void + listStudentCourses(studentId : int64) : list<Course> + listCourseAssignments(courseId : int64) : list<Assignment>

Interface (from diagram)	Operations and signatures (unchanged)
«interface» ISubmissionRepository	+ findFor(studentId : int64, assignmentId : int64) : optional<Submission> + findById(submissionId : int64) : optional<Submission> + createSubmissionAndVersion(submission : Submission, version : SubmissionVersion) : int64 + appendVersion(submissionId : int64, version : SubmissionVersion, newState : SubmissionState) : void + latestVersion(submissionId : int64) : SubmissionVersion + getVersion(versionId : int64) : SubmissionVersion + listForAssignment(assignmentId : int64) : list<Submission>
«interface» IAssessmentRepository	+ findById(resultId : int64) : optional<AssessmentResult> + findForSubmission(submissionId : int64) : optional<AssessmentResult> + listDepartmentResults(hodId : int64) : list<AssessmentResult> + listReleasedForStudent(studentId : int64) : list<AssessmentResult> + saveMarkAndAudit(result : AssessmentResult, audits : list<MarkChangeAudit>) : void + saveApproval(result : AssessmentResult, audits : list<MarkChangeAudit>) : void + saveRelease(resultId : int64, releasedAt : DateTime) : void + listAuditEvents(resultId : int64) : list<MarkChangeAudit>
«interface» ISubmissionContentStore	+ stage(sourcePath : string) : StagedRef + commit(staged : StagedRef) : StoredRef + discard(staged : StagedRef) : void + remove(stored : StoredRef) : void + read(stored : StoredRef) : Bytes